```
  it "can constrain the order messages were received in" do
    sequence = []

    allow(test_double).to receive(:message_1) do
      # TODO
    end

    allow(test_double).to receive(:message_2) do
      # TODO
    end

    test_double.message_1
    test_double.message_2
    test_double.message_1

    expect(sequence).to eq([:message_1, :message_2, :message_1])
  end
end
```

Your task is to fill in the body of each block (marked with `# TODO`) to make the specs pass.

Fluent Interface

Your solution so far has shown off how flexible your test doubles can be when you use a block implementation. Blocks are general enough to provide any behavior you need.

Now, you're going to explore RSpec's fluent interface for test doubles. Each of the blocks you wrote in the first exercise has a simpler rspec-mocks equivalent. Use the APIs you learned in this chapter to replace each block with a simpler `allow` or `expect` expression. For two of the constraint examples, you'll need to edit or remove some of the surrounding code, too.